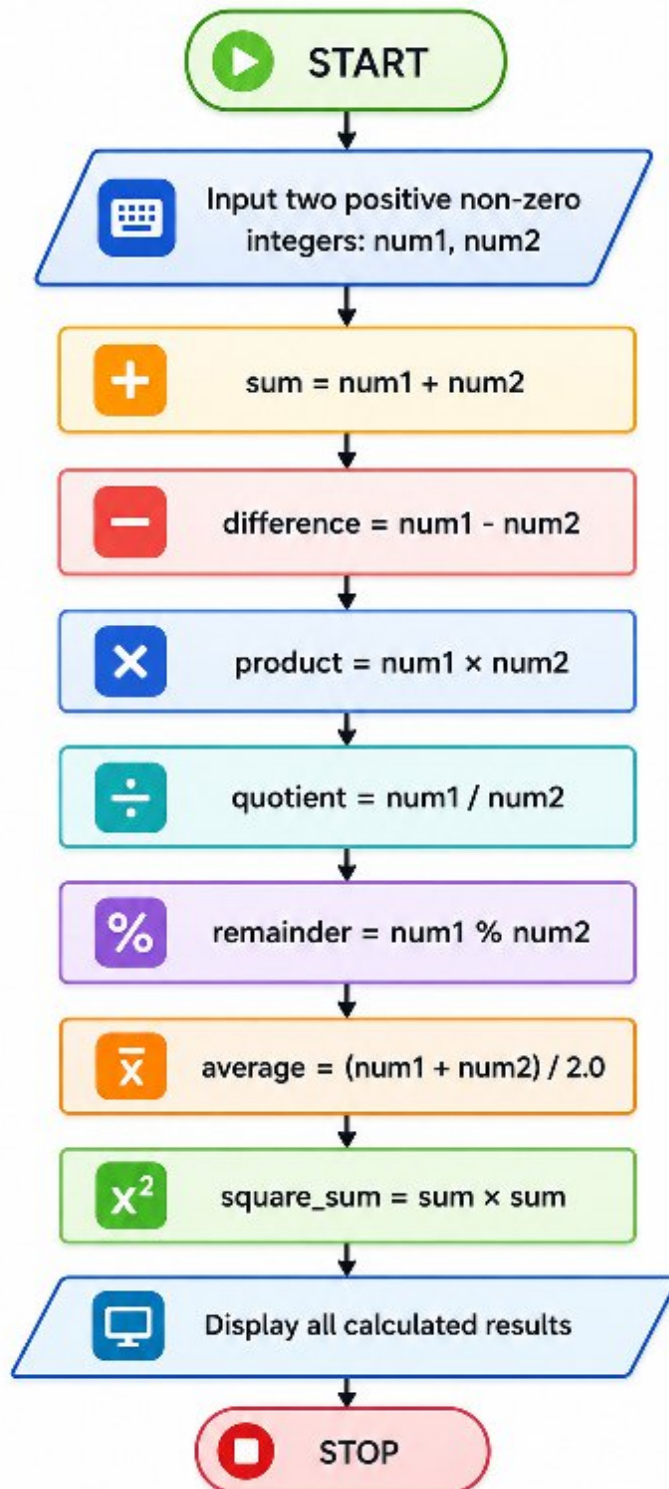


Q.1) //Sequential: Flowchart, Pseudocode, Algorithm, Code
Write a C program to input two non-zero positive integers and display their sum, difference, product, quotient, remainder, average, and square of their sum.

1. Flowchart



2. Pseudocode

BEGIN

INPUT num1, num2

sum = num1 + num2

difference = num1 – num2

product = num1 * num2

quotient = num1 / num2

remainder = num1 % num2

average = (num1 + num2) / 2.0

square_sum = sum * sum

DISPLAY sum

DISPLAY difference

DISPLAY product

DISPLAY quotient

DISPLAY remainder

DISPLAY average

DISPLAY square_sum

END

3. Algorithm

Algorithm: To Perform Arithmetic Operations on Two Positive Non-Zero Integers

1. Start.
2. Declare variables num1 and num2.
3. Declare variables to store sum, difference, product, quotient, remainder, average, and square of sum.
4. Input two non-zero positive integers num1 and num2.

5. Calculate the sum of the two numbers.
6. Calculate the difference of the two numbers.
7. Calculate the product of the two numbers.
8. Calculate the quotient using integer division.
9. Calculate the remainder using the modulus operator.
10. Calculate the average of the two numbers.
11. Calculate the square of their sum.
12. Display all the calculated results.
13. Stop.

4. C Code

```
#include <stdio.h>

int main()
{
    int num1, num2;
    int sum, difference, product;
    int quotient, remainder;
    float average;
    int square_sum;
    printf("Enter two non-zero positive integers: ");
    scanf("%d %d", &num1, &num2);
    sum = num1 + num2;
    difference = num1 - num2;
    product = num1 * num2;
    quotient = num1 / num2;
    remainder = num1 % num2;
    average = (num1 + num2) / 2.0;
    square_sum = sum * sum;
    printf("\nSum = %d", sum);
    printf("\nDifference = %d", difference);
```

```

printf("\nProduct = %d", product);
printf("\nQuotient = %d", quotient);
printf("\nRemainder = %d", remainder);
printf("\nAverage = %.2f", average);
printf("\nSquare of Sum = %d\n", square_sum);

return 0;
}

```

```

#include <stdio.h>

int main()
{
    int num1, num2;
    int sum, difference, product;
    int quotient, remainder;
    float average;
    int square_sum;

    printf("Enter two non-zero positive integers: ");
    scanf("%d %d", &num1, &num2);

    sum = num1 + num2;
    difference = num1 - num2;
    product = num1 * num2;
    quotient = num1 / num2;
    remainder = num1 % num2;
    average = (num1 + num2) / 2.0;
    square_sum = sum * sum;

    printf("\nSum = %d", sum);
    printf("\nDifference = %d", difference);
    printf("\nProduct = %d", product);
    printf("\nQuotient = %d", quotient);
    printf("\nRemainder = %d", remainder);
    printf("\nAverage = %.2f", average);
    printf("\nSquare of Sum = %d\n", square_sum);

    return 0;
}

```

```

kali@kali:~/denoi
$ gcc sequential.c

(gedit:45818): WARNING **: 02:11:11.483: Could not load Pcap repository: Typelib file for namespace 'Pcap', version '1.0' not found
(gedit:45818): WARNING **: 02:11:11.483: Could not load PcapGtk repository: Typelib file for namespace 'PcapGtk', version '1.0' not found
kali@kali:~/denoi
$ cc sequential.c
$ ./s.out
Enter two non-zero positive integers: 10
3

Sum = 13
Difference = 7
Product = 30
Quotient = 3
Remainder = 1
Average = 6.50
Square of Sum = 169

```

Q.2) //Conditional Branching: Flowchart, Pseudocode, Algorithm, Code

Write a menu-driven C program to input a temperature in Celsius and convert it into Kelvin, Fahrenheit, Reaumur, or Rankine based on the user's choice. Use simple IF statements to perform the selected conversion and display the result.

1. Flowchart

START

Input temperature in
Celsius (°C)

Display Conversion
Menu

1. Kelvin
2. Fahrenheit
3. Reaumur
4. Rankine

Enter choice

choice == 1 ?

YES NO

$$K = C + 273.15$$

choice == 2 ?

YES NO

$$F = (9/5)C + 32$$

choice == 3 ?

YES NO

$Re = (4/5)C$

choice == 4 ?

YES NO

$Ra = (C + 273.15) \times \frac{9}{5}$ Invalid Choice

Display conversion
result

STOP

Evaluation of the Flowchart Against Test Cases

The flowchart was evaluated with different temperatures and menu choices.

For example, for **25°C with choice 2**, the flowchart follows:

START → Input 25°C → Display Menu → Choice 2 → Fahrenheit calculation → Display 77.00°F → STOP

For an invalid choice such as 5, all four conditions are false, so the flowchart displays **Invalid Choice**.

Thus, **the flowchart correctly represents the algorithm and successfully handles all the prepared test cases.**

2. Pseudocode

```
BEGIN
```

```
INPUT celsius
```

```
DISPLAY "1. Celsius to Kelvin"
```

```
DISPLAY "2. Celsius to Fahrenheit"
```

```
DISPLAY "3. Celsius to Reaumur"
```

```
DISPLAY "4. Celsius to Rankine"
```

```
INPUT choice
```

```
IF choice == 1 THEN
```

```
    result = celsius + 273.15
```

```
    DISPLAY result, "Kelvin"
```

```
END IF
```

```
IF choice == 2 THEN
```

```
    result = (9.0 / 5.0) * celsius + 32
```

```
    DISPLAY result, "Fahrenheit"
```

```
END IF
```

```
IF choice == 3 THEN
```

```
    result = (4.0 / 5.0) * celsius
```

```
    DISPLAY result, "Reaumur"
```

```
END IF
```

```
IF choice == 4 THEN
```

```
    result = (celsius + 273.15) * (9.0 / 5.0)
```

```
    DISPLAY result, "Rankine"
```

```
END IF
```

```
IF choice < 1 OR choice > 4 THEN
```

```
    DISPLAY "Invalid Choice"
```

```
END IF
```

```
END
```

3) Code

```
#include <stdio.h>
```

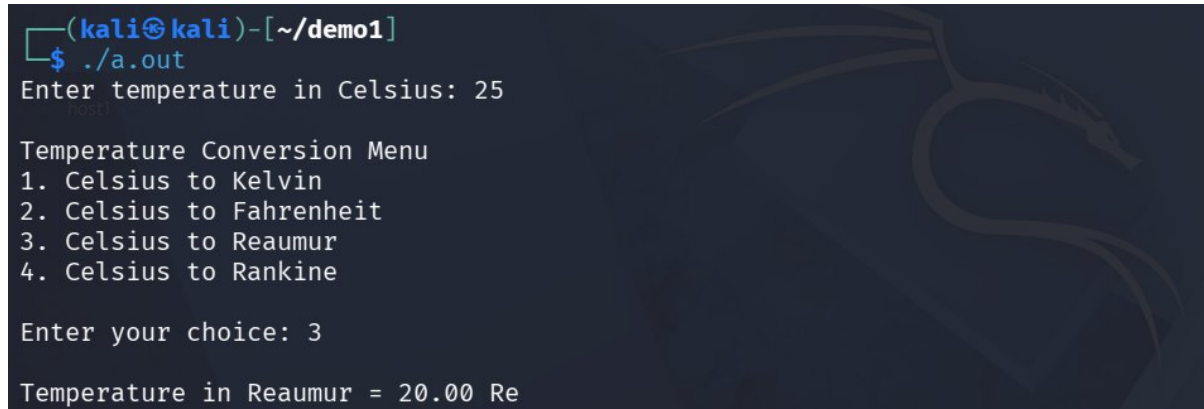
```
int main()
```

```
{
```

```
    float celsius, result;
    int choice;
    printf("Enter temperature in Celsius: ");
    scanf("%f", &celsius);
    printf("\nTemperature Conversion Menu\n");
    printf("1. Celsius to Kelvin\n");
    printf("2. Celsius to Fahrenheit\n");
    printf("3. Celsius to Reaumur\n");
    printf("4. Celsius to Rankine\n");
    printf("\nEnter your choice: ");
    scanf("%d", &choice);
    if (choice == 1)
    {
        result = celsius + 273.15;
        printf("\nTemperature in Kelvin = %.2f K\n", result);
    }
    if (choice == 2)
    {
        result = (9.0 / 5.0) * celsius + 32;
        printf("\nTemperature in Fahrenheit = %.2f F\n", result);
    }
    if (choice == 3)
    {
        result = (4.0 / 5.0) * celsius;
        printf("\nTemperature in Reaumur = %.2f Re\n", result);
    }
    if (choice == 4)
    {
        result = (celsius + 273.15) * (9.0 / 5.0);
        printf("\nTemperature in Rankine = %.2f Ra\n", result);
    }
}
```



```
if (choice < 1 || choice > 4)
{
    printf("\nInvalid choice!\n");
}
return 0;
}
```



```
(kali㉿kali)-[~/demo1]
$ ./a.out
Enter temperature in Celsius: 25

Temperature Conversion Menu
1. Celsius to Kelvin
2. Celsius to Fahrenheit
3. Celsius to Reaumur
4. Celsius to Rankine

Enter your choice: 3

Temperature in Reaumur = 20.00 Re
```

4) Algorithm

Algorithm: Temperature Conversion Using Conditional Branching

1. Start.
2. Declare variables celsius, result, and choice.
3. Input the temperature in Celsius.
4. Display the conversion menu.
5. Input the user's choice.
6. If the choice is 1, convert Celsius to Kelvin using:
 $K = C + 273.15$
7. If the choice is 2, convert Celsius to Fahrenheit using:
 $F = (9/5) \times C + 32$
8. If the choice is 3, convert Celsius to Reaumur using:
 $Re = (4/5) \times C$
9. If the choice is 4, convert Celsius to Rankine using:
 $Ra = (C + 273.15) \times 9/5$
10. If none of the choices from 1 to 4 is selected, display **Invalid**

choice.

11. Display the converted temperature.
12. Stop.

Evaluation of the Algorithm Against Test Cases

The algorithm was evaluated using the prepared test cases.

For Test Case , the input temperature is **25°C** and the choice is **1**. Since the choice is 1, the Kelvin conversion is selected:

$$\mathbf{K = 25 + 273.15 = 298.15\ K}$$

This matches the expected output.

Similarly:

Choice 2 gives **77.00°F**

Choice 3 gives **20.00°Re**

Choice 4 gives **536.67°Ra**

The algorithm also handles an invalid choice such as 5 by displaying an appropriate invalid-choice message.

Therefore, **the algorithm successfully passes all the prepared test cases.**

Q.3) //Iterative Looping: Flowchart, Pseudocode, Algorithm, Code Write a C program to input a positive integer and find the number of digits using the formula "digits = log10(num) + 1". Using a FOR loop, extract each digit of the number and display the sum and product of its digits

1. Algorithm

Algorithm: To Find the Number of Digits, Sum and Product of Digits

1. Start.
2. Declare variables num, temp, digit, digits, sum, product, and i.
3. Input a positive integer num.
4. Calculate the number of digits using:

$$\mathbf{digits = log10(num) + 1}$$

5. Store the value of num in temp so that the original number can be preserved.

6. Initialize sum = 0.
7. Initialize product = 1.
8. Use a FOR loop from i = 1 to digits.
9. Extract the last digit using:

digit = temp % 10

10. Add the extracted digit to sum.
11. Multiply the extracted digit with product.
12. Remove the last digit using:

temp = temp / 10

13. Repeat the loop until all digits have been processed.
14. Display the number of digits.
15. Display the sum of the digits.
16. Display the product of the digits.
17. Stop.

Evaluation of the Algorithm Against Test Cases

The algorithm was evaluated using the prepared test cases.

For example, consider **1234**:

Number of digits

Iteration	Extracted Digit	Sum	Product
1	4	4	4
2	3	7	12
3	2	9	24
4	1	10	24

Therefore:

Number of digits = **4**

Sum = **10**

Product = **24**

The results match the expected output.

The algorithm also correctly handles numbers containing zero. For example, for 1005, the product becomes 0 because one of its digits is zero.

Therefore, **the algorithm successfully passes all the prepared test cases.**

2. Flowchart

START

Input positive
integer num

$\text{digits} = \log_{10}(\text{num}) + 1$

temp = num
sum = 0
product = 1

i < digits ?

YES

NO

$\text{digit} = \text{temp} \% 10$

$\text{sum} = \text{sum} + \text{digit}$

$\text{product} = \text{product}$
 $\times \text{digit}$

temp = temp / 10

i = i + 1

Display digits,
sum and product

STOP

Evaluation of the Flowchart Against Test Cases

For input 1234, the flowchart calculates:

digits = 4

The loop then executes four times and extracts:

4 → 3 → 2 → 1

The resulting:

Sum = 10

Product = 24

matches the expected output.

For 1005, the extracted digits are:

5 → 0 → 0 → 1

giving:

Sum = 6

Product = 0

Thus, **the flowchart correctly represents the algorithm and passes all the prepared test cases.**

3. Pseudocode

BEGIN

```
INPUT num
```

```
digits = log10(num) + 1
```

```
temp = num
```

```
sum = 0
```

```
product = 1
```

```
FOR i = 1 TO digits
```

```
    digit = temp % 10
```

```
    sum = sum + digit
```

```
    product = product * digit
```

```
    temp = temp / 10
```

```
END FOR
```

```
DISPLAY digits
```

```
DISPLAY sum
```

```
DISPLAY product
```

```
END
```

4. C Program

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main()
```

```
{
```

```
    int num, temp, digit;
```

```
    int digits, sum = 0, product = 1;
```

```
    int i;
```

```
printf("Enter a positive integer: ");
scanf("%d", &num);

digits = (int)log10(num) + 1;

temp = num;

for (i = 1; i <= digits; i++)
{
    digit = temp % 10;

    sum = sum + digit;

    product = product * digit;

    temp = temp / 10;
}

printf("\nNumber of digits = %d", digits);
printf("\nSum of digits = %d", sum);
printf("\nProduct of digits = %d\n", product);

return 0;
}
```

Name - Reyansh Pandit

SAP ID - 590033602

Batch - 16